

R documentation

of ‘man/BayRC-package.Rd’ etc.

July 14, 2026

CB_init_single	<i>Initialize single-species BayRC MCMC chain</i>
----------------	---

Description

Produces starting values for the BayRC MCMC sampler by fitting an ordinary least-squares cosinor model to every gene. OLS-derived MESOR, amplitude, phase, and residual variance are used as initial values; rhythmicity is initialised from an F-test at $\alpha = 0.05$. When `FitCosinor = FALSE`, parameters are drawn from their priors.

Usage

```
CB_init_single(  
  Data.list = list(data = as.data.frame(a.sim.dat$data[[1]]$dat), time =  
    a.sim.dat$data[[1]]$x$time, gname = paste0("G",  
      seq_len(nrow(a.sim.dat$data[[1]]$dat)))),  
  P = 24,  
  FitCosinor = TRUE,  
  mu_M = 0,  
  sigma_M = 10,  
  mu_A = 1,  
  sigma_A = 10,  
  seed = 15213  
)
```

Arguments

<code>Data.list</code>	A named list with three elements: <code>data</code> (G x N data.frame of expression values), <code>time</code> (length-N numeric Zeitgeber time vector in hours), and <code>gname</code> (length-G character vector of gene identifiers).
<code>P</code>	Numeric; circadian period in hours (default 24).
<code>FitCosinor</code>	Logical; if TRUE (default) use OLS cosinor estimates; if FALSE draw from priors.
<code>mu_M</code>	Numeric; prior mean for MESOR (default 0).
<code>sigma_M</code>	Numeric; prior standard deviation for MESOR (default 10).

<code>mu_A</code>	Numeric; prior mean for amplitude (default 1).
<code>sigma_A</code>	Numeric; prior standard deviation for amplitude (default 10).
<code>seed</code>	Integer; random seed (default 15213).

Details

Initialize MCMC chain for a single circadian dataset

Value

A named list with elements `rho` (length-G logical), `M` (length-G MESOR), `A` (length-G amplitude), `phi` (length-G acrophase in hours), and `sigma` (length-G residual variance). Suitable for passing directly to `CB_MCMC_single_rj_slice` as `Init.value`.

Examples

```
## Not run:
sim <- CBt_sim_data()
dat <- list(data = as.data.frame(sim$data[[1]]$dat),
            time = sim$data[[1]]$x$time,
            gname = paste0("G", seq_len(nrow(sim$data[[1]]$dat))))
init <- CB_init_single(dat)

## End(Not run)
```

<code>CBt_init_single</code>	<i>Initialize time-error BayRC MCMC chain</i>
------------------------------	---

Description

Produces starting values for the CBt MCMC sampler, which models Zeitgeber-time measurement error (`t_p`). Parameter initialisation follows the same OLS cosinor approach as `CB_init_single` but also initialises the time-error latent variable `t_p` to zero.

Usage

```
CBt_init_single(
  Data.list = list(data = as.data.frame(a.sim.dat$data[[1]]$dat), time =
    a.sim.dat$data[[1]]$x$time, gname = paste0("G",
    seq_len(nrow(a.sim.dat$data[[1]]$dat)))),
  P = 24,
  FitCosinor = TRUE,
  mu_M = 0,
  sigma_M = 10,
  mu_A = 1,
  sigma_A = 10,
  seed = 15213
)
```

Arguments

<code>Data.list</code>	A named list with three elements: <code>data</code> (G x N data.frame of expression values), <code>time</code> (length-N numeric Zeitgeber time vector in hours), and <code>gname</code> (length-G character gene identifiers).
<code>P</code>	Numeric; circadian period in hours (default 24).
<code>FitCosinor</code>	Logical; if TRUE (default) use OLS cosinor estimates; if FALSE draw from priors.
<code>mu_M</code>	Numeric; prior mean for MESOR (default 0).
<code>sigma_M</code>	Numeric; prior standard deviation for MESOR (default 10).
<code>mu_A</code>	Numeric; prior mean for amplitude (default 1).
<code>sigma_A</code>	Numeric; prior standard deviation for amplitude (default 10).
<code>seed</code>	Integer; random seed (default 15213).

Details

Initialize MCMC chain for a dataset with Zeitgeber-time uncertainty

Value

A named list with elements `rho`, `M`, `A`, `phi`, `sigma` (same as `CB_init_single`), plus `t_p` (length-N vector of initial time-error values, set to 0). Suitable for passing to `CBt_MCMC_single` as `Init.value`.

Examples

```
## Not run:
sim <- CBt_sim_data()
dat <- list(data = as.data.frame(sim$data[[1]]$dat),
           time = sim$data[[1]]$x$time,
           gname = paste0("G", seq_len(nrow(sim$data[[1]]$dat))))
init <- CBt_init_single(dat)

## End(Not run)
```

CB_MCMC_single_rj_slice

CB MCMC single reversible-jump slice sampler

Description

Runs the BayRC RJMCMC sampler for a single-species/condition circadian expression dataset. Within each iteration the sampler proposes model-space jumps (`rho`: 0/1 rhythmicity indicator) via an RJMCMC acceptance ratio and updates amplitude, phase, MESOR, and residual variance via Gibbs or slice steps.

Usage

```

CB_MCMC_single_rj_slice(
  Data.list,
  Init.value,
  P = 24,
  iteration = 3000,
  thin = 20,
  n.burn = 1000,
  seed = 15213,
  diagnostics = FALSE,
  p_rhythmic = rep(0.2, 100),
  rj.p.stay = 0.5,
  A_prior = "Jeffreys_OLS_condi",
  mu_A = 1,
  sigma_A = 10^2,
  A.min = 0,
  A_wb_beta2 = 2,
  A_gm_shape = 1.99,
  A_gm_rate = 0.5,
  rj.phi = TRUE,
  rj.A = TRUE,
  mu_M = 0,
  sigma_M = 10^2,
  sigma_prior_v = 4,
  sigma_prior_s = 1,
  save.file = "MCMC_save.rds",
  save.file2 = "MCMC_save.rds"
)

```

Arguments

<code>Data.list</code>	A named list with three elements: <code>data</code> (G x N data.frame of expression values), <code>time</code> (length-N numeric Zeitgeber time vector in hours), and <code>gname</code> (length-G character vector of gene identifiers).
<code>Init.value</code>	List returned by <code>CB_init_single</code> containing starting values for <code>rho</code> , <code>M</code> , <code>A</code> , <code>phi</code> , and <code>sigma</code> .
<code>P</code>	Numeric; circadian period in hours (default 24).
<code>iteration</code>	Integer; total number of MCMC iterations including burn-in (default 3000).
<code>thin</code>	Integer; thinning interval; every <code>thin</code> -th post-burn-in sample is stored (default 20).
<code>n.burn</code>	Integer; number of burn-in iterations discarded before storing samples (default 1000).
<code>seed</code>	Integer; random seed for reproducibility (default 15213).
<code>p_rhythmic</code>	Numeric vector of length G; prior probability $\Pr(\rho_g = 1)$ for each gene. Enters the RJMCMC log-prior odds as $\log(p / (1 - p))$. Default is <code>rep(0.2, 100)</code> .
<code>rj.p.stay</code>	Numeric in (0, 1); probability of skipping the between-model move (staying in the current model) at each iteration (default 0.5).

<code>A_prior</code>	Character; prior on amplitude. One of "Jeffreys_OLS_condi", "trunc_Normal", "Jeffreys", "Jeffreys_ridge", "sq_expo", or "gamma" (default "Jeffreys_OLS_condi").
<code>mu_A</code>	Numeric; mean of the truncated-Normal amplitude prior when <code>A_prior = "trunc_Normal"</code> (default 1).
<code>sigma_A</code>	Numeric; variance of the truncated-Normal amplitude prior (default 10^2).
<code>A.min</code>	Numeric; lower truncation bound for amplitude (default 0).
<code>A_wb_beta2</code>	Numeric; beta parameter for the squared-exponential amplitude prior <code>sq_expo</code> (default 2).
<code>A_gm_shape</code>	Numeric; shape parameter for the gamma amplitude prior; values ≤ 2 give a flatter prior (default 1.99).
<code>A_gm_rate</code>	Numeric; rate parameter for the gamma amplitude prior; smaller values give a flatter prior (default 0.5).
<code>rj.phi</code>	Logical; if TRUE, phase phi is jointly proposed in the RJMCMC birth/death step (default TRUE).
<code>rj.A</code>	Logical; if TRUE, amplitude A is jointly proposed in the RJMCMC birth/death step (default TRUE).
<code>mu_M</code>	Numeric; prior mean for MESOR (default 0).
<code>sigma_M</code>	Numeric; prior variance for MESOR (default 10^2).
<code>sigma_prior_v</code>	Numeric; degrees-of-freedom hyperparameter of the inverse-gamma prior on residual variance (default 4).
<code>sigma_prior_s</code>	Numeric; scale hyperparameter of the inverse-gamma prior on residual variance (default 1).
<code>save.file</code>	Character; path for saving intermediate MCMC output (default "MCMC_save.rds").
<code>save.file2</code>	Character; secondary save path used inside the RJMCMC helper (default "MCMC_save.rds").

Details

Run reversible-jump MCMC for a single circadian dataset

Value

A named list of $G \times K$ matrices ($K = \text{floor}(\text{iteration} / \text{thin})$):

rho Posterior samples of the rhythmicity indicator in $\{0, 1\}$.

phi Posterior samples of acrophase in hours (Zeitgeber scale).

A Posterior samples of amplitude.

M Posterior samples of MESOR.

sigma Posterior samples of residual variance.

log.r1, log.r1.SS, log.r3_A, log.r3_phi RJMCMC log-ratio components (for diagnostics).

if.accept.rj Binary matrix indicating accepted RJMCMC moves.

All matrices have rownames equal to `Data.list$gname`. After calling `match_symbols()`, the attributes `attr(rho, "symbols")` and `attr(rho, "RHYindex")` are set.

Examples

```
## Not run:
sim <- CBt_sim_data()
dat <- list(data = as.data.frame(sim$data[[1]]$dat),
            time = sim$data[[1]]$x$time,
            gname = paste0("G", seq_len(nrow(sim$data[[1]]$dat))))
init <- CB_init_single(dat)
mcmc_out <- CB_MCMC_single_rj_slice(dat, init,
                                   iteration = 200, thin = 10, n.burn = 100)

## End(Not run)
```

CB_getAllEst

Get all posterior estimates from BayRC MCMC output

Description

Computes posterior estimates and highest-density credible intervals for amplitude (A), acrophase (phi), MESOR (M), residual variance (sigma), and optionally peak time (t_p) from the MCMC output list. Circular statistics (`circular_median` and `circular_HDI`) are used for phi/t_p; linear posterior means and HDI are used for the rest.

Usage

```
CB_getAllEst(
  res,
  burn = 100,
  CI = TRUE,
  credMass = 0.95,
  P = 24,
  rhythmic = FALSE
)
```

Arguments

<code>res</code>	Named list; MCMC output from <code>CB_MCMC_single_rj_slice</code> (or <code>CBt_MCMC_single</code>). Must contain matrices <code>rho</code> , <code>A</code> , <code>phi</code> , <code>M</code> , <code>sigma</code> with <code>attr(rho, "symbols")</code> and <code>attr(rho, "RHYindex")</code> set by <code>match_symbols</code> .
<code>burn</code>	Integer; number of initial MCMC iterations to discard as additional burn-in when computing estimates (default 100).
<code>CI</code>	Logical; if <code>TRUE</code> (default), return lower and upper credible-interval bounds in addition to the point estimate.
<code>credMass</code>	Numeric in (0, 1); nominal coverage of the HDI (default 0.95).
<code>P</code>	Numeric; period in hours used for circular statistics (default 24).
<code>rhythmic</code>	Logical; if <code>TRUE</code> , return only rows for genes classified as rhythmic (<code>RHYindex == 1</code> ; default <code>FALSE</code>).

Details

Extract posterior estimates and credible intervals for all parameters

Value

A list of data.frames, one per parameter, each with columns `<param>.Est`, `<param>.Lower`, `<param>.Upper` and `RHYindex`. Row names are gene symbols when `attr(rho, "symbols")` is set.

Examples

```
## Not run:
sim <- CBt_sim_data()
dat <- list(data = as.data.frame(sim$data[[1]]$dat),
            time = sim$data[[1]]$x$time,
            gname = paste0("G", seq_len(nrow(sim$data[[1]]$dat))))
init <- CB_init_single(dat)
res <- CB_MCMC_single_rj_slice(dat, init,
                              iteration = 300, thin = 10, n.burn = 100)
est <- CB_getAllEst(res, burn = 10)

## End(Not run)
```

CBt_sim_data

*Simulate CBt circadian dataset***Description**

Generates synthetic multi-group circadian expression data following the BayRC cosinor model, optionally with Zeitgeber-time measurement error (`t_p`) on a specified proportion of samples. Each gene's true phase is drawn uniformly from $[0, P)$, and expression is generated as $Y = M + A * \cos(\omega * (t + t_p) - \phi) + \epsilon$. Supports multiple rhythmicity/phase-shift scenarios defined by `TOJR.grid`.

Usage

```
CBt_sim_data(
  P = 24,
  TOJR.grid = NULL,
  dParam = NULL,
  G = NULL,
  n = 30,
  fixed_tp = TRUE,
  sigma.t = 2,
  n.sigma.t = 16,
  p.sigma.t = NULL,
  Params = list(A = 0.5, phi = 0, M = 5, sigma = 1),
  TimePoints = NULL,
  PhaseClust = NULL
)
```

Arguments

<code>P</code>	Numeric; circadian period in hours (default 24).
<code>TOJR.grid</code>	Data.frame or matrix; <code>G_scenario</code> x <code>n.group</code> binary rhythmicity indicator grid defining which groups are rhythmic for each scenario.

dParam	Named list; group-level parameter offsets (dA, dPhase, dM) relative to the reference group, one element per non-reference group.
G	Integer vector of length n_scenario; number of genes per scenario row.
n	Integer; number of samples (default 30).
fixed_tp	Logical; if TRUE (default), time errors are fixed at +/- sigma.t; if FALSE, they are drawn from N(0, sigma.t).
sigma.t	Numeric; magnitude of the Zeitgeber-time measurement error (default 2).
n.sigma.t	Integer or NULL; number of samples with non-zero t_p. Exactly one of n.sigma.t and p.sigma.t must be non-NULL.
p.sigma.t	Numeric or NULL; proportion of samples with non-zero t_p.
Params	Named list; base cosinor parameters: A (amplitude), phi (phase), M (MESOR), sigma (residual SD).
TimePoints	Numeric vector or NULL; fixed observation times; if NULL, times are drawn uniformly from \[0, P).
PhaseClust	Currently reserved (default NULL).

Details

Simulate circadian expression data with Zeitgeber-time measurement error

Value

A named list with elements:

data A list of length n.group; each element contains x (sample-level covariates including time, time.p), beta (gene-level true parameters), and dat (G x N expression matrix).

TOJR Data.frame; expanded rhythmicity grid concatenated with parameter offsets for all genes.

Examples

```
## Not run:
sim <- CBT_sim_data(n.sigma.t = 8)

## End(Not run)
```

circular_HDI

Compute the shortest circular highest-density interval

Description

Finds the shortest interval on a circle of circumference P that contains at least credMass of the posterior samples. Uses a sliding-window algorithm on a doubled sorted array to handle wrap-around.

Usage

```
circular_HDI(samples, credMass = 0.95, P = 24, a = 0)
```

Arguments

<code>samples</code>	Numeric vector; phase samples in hours.
<code>credMass</code>	Numeric; desired coverage probability (default 0.95).
<code>P</code>	Numeric; period / circle circumference in hours (default 24).
<code>a</code>	Numeric; left endpoint of the normalisation interval (default 0).

Value

List with elements `lower` and `upper` (both in $[a, a + P)$) giving the bounds of the shortest HDI.

<code>circular_median</code>	<i>Compute the circular median of phase samples</i>
------------------------------	---

Description

Converts samples from hours to radians, computes the circular median via `circular::median.circular`, and converts back to hours.

Usage

```
circular_median(samples, P = 24)
```

Arguments

<code>samples</code>	Numeric vector; phase samples in hours.
<code>P</code>	Numeric; period in hours (default 24).

Value

Numeric scalar; circular median in hours in $[0, P)$.

<code>match_symbols</code>	<i>Map Ensembl IDs to gene symbols and set rhythmicity attributes</i>
----------------------------	---

Description

Takes the raw list output from `CB_MCMC_single_rj_slice` and (optionally) converts Ensembl row names to HGNC gene symbols via `biomaRt`. If row names are already symbols they are used directly. Computes a Bayes Factor for each gene using `summarize_bay` and sets `attr(rho, "symbols")` and `attr(rho, "RHYindex")` so that downstream functions can access gene identifiers and rhythmicity classifications. Duplicate symbols are resolved by keeping the row with the highest Bayes Factor.

Usage

```
match_symbols(input_df, BF, p_rhythmic = 0.5, ensemble = NULL)
```

Arguments

input_df	Named list; raw MCMC output (e.g. from <code>CB_MCMC_single_rj_slice</code>). Must contain a <code>rho</code> matrix with gene identifiers as row names.
BF	Numeric; Bayes Factor threshold above which a gene is called rhythmic.
p_rhythmic	Numeric in (0, 1); prior probability of rhythmicity used to compute the Bayes Factor (default 0.5).
ensemble	A biomaRt Mart object; required when row names are Ensembl IDs (default <code>NULL</code>).

Details

Annotate MCMC output with gene symbols and rhythmicity index

Value

The input list filtered to unique symbols, with all matrix elements row-subsetted and `attr(*, "symbols")`, `attr(*, "RHYindex")`, and (if Ensembl IDs were present) `attr(*, "ensembl_gene_ids")` set on each matrix.

Examples

```
## Not run:
res <- CB_MCMC_single_rj_slice(dat, init)
res2 <- match_symbols(res, BF = 3, p_rhythmic = 0.2)

## End(Not run)
```

```
bfdr_from_posterior
```

BFDR threshold from posterior rhythmicity probabilities

Description

Implements the BFDR rule described in the BayRC paper (Eq. 25): sort genes by decreasing posterior probability `p_g`, compute the running average of $(1 - p_g)$, and return the largest threshold `tau_c` such that $\text{BFDR}(\text{tau}_c) = \text{mean}(1 - p_g \mid p_g \geq \text{tau}_c) \leq \alpha$. This controls the expected proportion of false rhythmic calls at level α .

Usage

```
bfdr_from_posterior(posterior_probs, alpha = 0.05)
```

Arguments

posterior_probs	Numeric vector; marginal posterior probability of rhythmicity for each gene (values in $[0, 1]$).
alpha	Numeric; desired BFDR level (default 0.05).

Details

Estimate the Bayesian False Discovery Rate threshold from posterior probabilities

Value

A list with elements:

threshold Numeric; the BFDR-optimal cutoff τ_c .

rhythmic_genes Logical vector; TRUE for called rhythmic.

n_rhythmic Integer; number of rhythmic calls.

bfdr_values Numeric vector; BFDR at each ranked position.

sorted_probs Sorted (descending) posterior probabilities.

detect_rhy

Detect rhythmic genes in two conditions using BFDR control

Description

Applies `bfdr_from_posterior` separately to the marginal posterior rhythmicity probabilities of two conditions and returns the BFDR-optimal rhythmic gene sets for each, together with data.frames of annotated gene information.

Usage

```
detect_rhy(dat1, dat2, bfdr_alpha = 0.05)
```

Arguments

dat1 Named list; MCMC output for condition A with a `rho` matrix.

dat2 Named list; MCMC output for condition B with a `rho` matrix.

bfdr_alpha Numeric; BFDR level (default 0.05).

Value

A list with elements:

rhythmic_A_logical, rhythmic_B_logical Logical vectors for conditions A and B.

rhythmic_A, rhythmic_B Data.frames of rhythmic genes with posterior probability columns.

threshold_A, threshold_B BFDR thresholds for each condition.

n_rhythmic_A, n_rhythmic_B, n_total, bfdr_alpha Counts and settings.

summarize_bay

Summarise posterior rhythmicity and Bayes Factor per gene

Description

Computes the posterior mean rhythmicity (`RowAverage`), the number of zero samples (`Zeroes`), and the Bayes Factor for each gene. The Bayes Factor is computed as posterior odds / prior odds: $BF = (\rho_{\text{bar}} / (1 - \rho_{\text{bar}})) / (p_{\text{rhythmic}} / (1 - p_{\text{rhythmic}}))$. Genes with $BF >$ the threshold are called rhythmic (`Rhythmicity = 1`).

Usage

```
summarize_bay(input_df, BF, p_rhythmic = 0.2)
```

Arguments

<code>input_df</code>	G x K binary matrix of posterior rho samples.
<code>BF</code>	Numeric; Bayes Factor threshold for calling a gene rhythmic.
<code>p_rhythmic</code>	Numeric; prior probability of rhythmicity (default 0.5).

Value

`Data.frame` with columns `Zeroes`, `RowAverage`, `Rhythmicity` (0/1), and `BayesF`.

transition_classify

Bayesian transition classification (gain / loss / maintained)

Description

Computes joint transition posterior probabilities: $p_{\text{gain}} = (1 - p_{\text{A}}) * p_{\text{B}}$ (gained rhythmicity), $p_{\text{loss}} = p_{\text{A}} * (1 - p_{\text{B}})$ (lost rhythmicity), $p_{\text{cons}} = p_{\text{A}} * p_{\text{B}}$ (maintained rhythmicity), then applies `bfd_r_from_posterior` to each to identify gain, loss, and conserved gene sets at the specified BFDR level.

Usage

```
transition_classify(pA, pB, bfd_r_alpha = 0.05)
```

Arguments

<code>pA</code>	Numeric vector; marginal posterior rhythmicity probability for condition A (reference; length G).
<code>pB</code>	Numeric vector; marginal posterior rhythmicity probability for condition B (length G).
<code>bfd_r_alpha</code>	Numeric; BFDR control level (default 0.05).

Details

Classify rhythmicity transitions using joint BFDR on transition posteriors

Value

A list with elements:

gain_loss_status Named character vector of length G with values "Gain", "Loss", "Maintained", or "Non-rhythmic".

gain_genes, loss_genes, cons_genes Integer indices of genes in each class.

tau_gain, tau_loss, tau_cons BFDR thresholds for each transition type.

n_gain, n_loss, n_cons Counts.

results Data.frame with per-gene classification details.

```
transition_classify_marginal
```

Classify rhythmicity transitions using marginal BFDR on each condition

Description

Applies `bfd_r_from_posterior` separately to `p_A` and `p_B` (marginal rhythmicity probabilities) to define `tau_A` and `tau_B`, then classifies genes as Gain, Loss, Maintained, or Non-rhythmic based on whether each gene exceeds its condition's threshold. Contrast with `transition_classify`, which uses joint transition probabilities.

Usage

```
transition_classify_marginal(pA, pB, bfd_r_alpha = 0.05)
```

Arguments

<code>pA</code>	Numeric vector; marginal posterior rhythmicity probability for condition A (length G).
<code>pB</code>	Numeric vector; marginal posterior rhythmicity probability for condition B (length G).
<code>bfd_r_alpha</code>	Numeric; BFDR control level (default 0.05).

Value

A list with the same structure as `transition_classify` but with `tau_A` and `tau_B` instead of `tau_gain/loss/cons`, and a `results` data.frame with columns `rhythmic_A` and `rhythmic_B`.

phase_infer	<i>Bayesian phase inference with BFDR-controlled shift/conservation flags</i>
-------------	---

Description

For genes classified as "Maintained" by `transition_classify` or `transition_classify_marginal`, computes the posterior distribution of the phase difference $\Delta\phi = \phi_1 - \phi_2$ (wrapped to $[-P/2, P/2]$), estimates the probability of a significant shift ($|\Delta\phi| \geq \text{shift}$), and applies BFDR control to flag significantly shifted or conserved genes. Optionally computes the full circular HDI of $\Delta\phi$ when `compute_hdi = TRUE`.

Usage

```
phase_infer(
  phi_matrix1,
  phi_matrix2,
  gain_loss_status,
  P = 24,
  credMass = 0.95,
  shift = 2,
  a = -P/2,
  bfdr_alpha = 0.05,
  compute_hdi = FALSE
)
```

Arguments

<code>phi_matrix1</code>	G x K matrix; posterior phase samples for condition 1.
<code>phi_matrix2</code>	G x K matrix; posterior phase samples for condition 2.
<code>gain_loss_status</code>	Named character vector of length G with values "Gain", "Loss", "Maintained", or "Non-rhythmic" (from <code>transition_classify</code>).
<code>P</code>	Numeric; period in hours (default 24).
<code>credMass</code>	Numeric; HDI coverage (default 0.95); only used when <code>compute_hdi = TRUE</code> .
<code>shift</code>	Numeric; phase-shift threshold in hours (default 4).
<code>a</code>	Numeric; left endpoint for circular normalisation (default $-P/2$).
<code>bfdr_alpha</code>	Numeric; BFDR control level (default 0.05).
<code>compute_hdi</code>	Logical; if TRUE, compute the circular HDI for each maintained gene (slower; default FALSE).

Details

Infer phase shifts and conservation among maintained rhythmic genes

Value

A list with per-gene vectors:

peak1, peak2 Circular median phases for each condition.

deltaPhi.Est, deltaPhi.Lower, deltaPhi.Upper Phase-difference median and HDI (filled only when `compute_hdi = TRUE`).

prob_shift, prob_conserved Posterior probabilities of shift and conservation.

flag_shift, flag_cons, flag_undetermined BFDR-significant flags.

BFDR_shift, BFDR_cons Running BFDR values.

pathSelect	<i>FGSEA-based pathway selection for circadian transitions</i>
------------	--

Description

Applies fast gene-set enrichment analysis (FGSEA) to a posterior gene-ranking metric derived from two BayRC MCMC outputs. Genes are ranked by their log-odds-ratio of gain vs loss, or by conservation, gain, loss, or union probability; pathways are tested for enrichment toward either end of this ranking. Pathway-level effect sizes and probabilistic gain/loss/conservation indices are added to the output.

Usage

```
pathSelect (
  mcmc.merge.list,
  pathway.list,
  dataset.names = NULL,
  ranking.method = c("log_odds_ratio", "union", "conserved", "gain", "loss"),
  score_type = c("std", "pos", "neg"),
  pathwaysize.lower.cut = 10,
  pathwaysize.upper.cut = 200,
  qvalue.cut = 0.05,
  epsilon = 0.001,
  n_top_genes = 5,
  nperm = 10000,
  nproc = 1,
  seed = 12345
)
```

Arguments

`mcmc.merge.list`

Named list of exactly 2 MCMC output lists; the first is condition A (reference) and the second is condition B. Each must have `attr(rho, "symbols")` set.

`pathway.list` Named list of character vectors; pathway gene sets.

`dataset.names`

Character vector of length 2 or `NULL`; labels for conditions A and B (default auto-detected from list names).

```

ranking.method      Character; gene-ranking metric: one of "log_odds_ratio" (default), "union",
                    "conserved", "gain", or "loss".
score_type          Character; fgsea score type: "std" (two-sided), "pos" (gain only), or "neg"
                    (loss only, inverts ranking).
pathwaysize.lower.cut      Integer; minimum pathway size (default 10).
pathwaysize.upper.cut      Integer; maximum pathway size (default 200).
qvalue.cut            Numeric; adjusted p-value significance cutoff (default 0.05).
epsilon              Numeric; small constant added to probabilities to avoid log(0) (default 0.001).
n_top_genes          Integer; number of top genes to report per category per pathway (default 5).
nperm                Integer; fgsea permutations (default 10000).
nproc                Integer; fgsea parallel processes (default 1).
seed                 Integer; random seed (default 12345).

```

Details

Select circadian-enriched pathways using FGSEA

Value

A list with elements:

results Data.frame of fgsea results (one row per pathway) augmented with Gain_Index, Loss_Index, Conserved_Index, Gain_Loss_Ratio_Arithmetic, expected counts, and top gene columns.

gene_rankings Data.frame of per-gene ranking metrics and posterior probabilities.

parameters List of analysis settings.

Examples

```

## Not run:
res <- pathSelect(list(human = human_res, mouse = mouse_res),
                  pathway.list = msig_pathways,
                  ranking.method = "log_odds_ratio")
head(res$results)

## End(Not run)

```

```
plot_pathway_integrated
```

Pathway-level expression heatmap

Description

Produces a ComplexHeatmap visualisation for a given pathway, showing posterior rhythmicity probabilities and phase information for both conditions side by side. Genes can be coloured by gain/loss/ maintained status from transition_classify output. The plot can show all genes, only rhythmic genes, or both (two panels).

Usage

```
plot_pathway_integrated(
  data1,
  data2,
  pathway_genes,
  pathway_name,
  phase_results,
  transition_results = NULL,
  group_names = c("Group1", "Group2"),
  save_path = NULL,
  n_bins = 24,
  versions = c("full", "rhythmic_only", "both")
)
```

Arguments

<code>data1</code>	Named list; MCMC output for condition 1 with <code>rho</code> and <code>phi</code> matrices.
<code>data2</code>	Named list; MCMC output for condition 2.
<code>pathway_genes</code>	Character vector; gene names in the pathway.
<code>pathway_name</code>	Character; pathway label used in the plot title.
<code>phase_results</code>	Output from <code>phase_infer</code> or a similar list with per-gene phase metrics.
<code>transition_results</code>	Output from <code>transition_classify</code> or <code>NULL</code> (default <code>NULL</code>); used to colour genes by gain/loss/maintained status.
<code>group_names</code>	Character vector of length 2; condition labels (default <code>c("Group1", "Group2")</code>).
<code>save_path</code>	Character or <code>NULL</code> ; file path for saving the plot PNG; if <code>NULL</code> the plot is drawn but not saved.
<code>n_bins</code>	Integer; number of bins for the phase colour wheel (default 24).
<code>versions</code>	Character; which version to produce: "full", "rhythmic_only", or "both" (default "full").

Details

Integrated pathway heatmap with rhythmicity and phase information

Value

Called for side effects; invisibly returns the heatmap object.

`multi_conservation_pathway`

Multi-condition pathway circadian conservation analysis

Description

Runs all pairwise pathway conservation comparisons among a list of MCMC outputs: computes observed pathway congruence scores, generates permutation null distributions, applies BH correction, and writes results to an Excel file.

Usage

```
multi_conservation_pathway(
  mcmc.merge.list,
  dataset.names,
  select.pathway.list,
  delta = 3,
  units = "hours",
  B = 1000,
  ncores = NULL,
  parallel = "auto",
  rounds = 1,
  save_intermediate = FALSE,
  intermediate_dir = "intermediate_results",
  output.dir = "Conservation_Pathway"
)
```

Arguments

<code>mcmc.merge.list</code>	Named list of MCMC output lists, one per condition.
<code>dataset.names</code>	Character vector; condition labels.
<code>select.pathway.list</code>	Named list of character vectors; pathway gene sets.
<code>delta</code>	Numeric; phase threshold (default 3).
<code>units</code>	Character; "hours" (default).
<code>B</code>	Integer; permutations per pair (default 1000).
<code>ncores</code>	Integer or NULL; parallel cores.
<code>parallel</code>	Logical or "auto".
<code>rounds</code>	Integer; rounds per pair.
<code>save_intermediate</code>	Logical; save intermediate results.
<code>intermediate_dir</code>	Character; directory for intermediate saves.
<code>output.dir</code>	Character; directory for Excel output (default "Conservation_Pathway").

Details

Pairwise pathway conservation analysis across multiple conditions

Value

Named list of pairwise pathway result data.frames. Also writes an Excel file to `output.dir`.

multi_conservation_pathway_bootstrap

Pathway-level bootstrap concordance (compatibility wrapper)

Description

Thin wrapper around `multi_conservation` retained for backward compatibility with analysis scripts that call `multi_conservation_pathway_bootstrap`. All computation is delegated to `multi_conservation`; the `delta` and `units` arguments are accepted but unused (the c-score is threshold-free by design; see paper Eq. 3).

Note: output Excel sheets are named "Results" and "Column_Definitions", not "congruence_index" as in the legacy Thien/ version. Update downstream `read.xlsx` calls accordingly.

Usage

```
multi_conservation_pathway_bootstrap(
  mcmc.merge.list,
  dataset.names,
  select.pathway.list,
  delta = 3,
  units = "hours",
  n_boot = 500,
  n_perm = 1000,
  output.dir = "Conservation_Pathway_Bootstrap",
  ...
)
```

Arguments

<code>mcmc.merge.list</code>	Named list of MCMC outputs.
<code>dataset.names</code>	Character vector of dataset labels.
<code>select.pathway.list</code>	Named list of pathway gene sets.
<code>delta</code>	Numeric; ignored (reserved for future phase-threshold concordance variant).
<code>units</code>	Character; ignored.
<code>n_boot</code>	Integer; bootstrap replicates (default 500).
<code>n_perm</code>	Integer; permutation replicates (default 1000).
<code>output.dir</code>	Character; directory for Excel output.
<code>...</code>	Additional arguments passed to <code>multi_conservation</code> .

Value

Same value as `multi_conservation`.

multi_conservation *Multi-dataset pairwise pathway concordance analysis*

Description

Runs all pairwise pathway bootstrap concordance analyses across a list of MCMC outputs and compiles results into Excel output. Wraps `bootstrap_conservation_pathway_within`.

Usage

```
multi_conservation(
  mcmc.merge.list,
  dataset.names,
  select.pathway.list,
  n_perm = 1000,
  n_boot = 500,
  alpha = 0.05,
  min.p = 5,
  qvalue_threshold = NULL,
  output.dir = "Conservation_Pathway_Bootstrap",
  save_output = TRUE,
  use_cpp = FALSE,
  compute_pvalue = TRUE,
  compute_ci = TRUE
)
```

Arguments

<code>mcmc.merge.list</code>	Named list of MCMC output lists.
<code>dataset.names</code>	Character vector; condition labels.
<code>select.pathway.list</code>	Named list of pathway gene sets.
<code>n_perm</code>	Integer; permutations per pathway (default 1000).
<code>n_boot</code>	Integer; bootstrap replicates per pathway (default 500).
<code>alpha</code>	Numeric; significance level (default 0.05).
<code>min.p</code>	Integer; minimum number of pathway genes required to test a pathway (default 5).
<code>qvalue_threshold</code>	Numeric or NULL; q-value significance cutoff.
<code>output.dir</code>	Character; directory for Excel output.

Value

Named list of pairwise result data.frames. Also writes an Excel file to `output.dir`.

match_homologs	<i>Match homologs across species MCMC outputs</i>
----------------	---

Description

Uses biomaRt to retrieve one-to-one orthologs between each non-reference species and the reference species (default human), then intersects the mapped gene sets across all datasets. Each input list is row-filtered and reordered so that all datasets share the same genes in the same order, enabling direct cross-species comparisons.

Usage

```
match_homologs(input_dfs, species_from, ref = "human")
```

Arguments

input_dfs	A list of MCMC output lists (one per dataset), each with matrices whose row names are Ensembl gene IDs and whose <code>attr(rho, "ensembl_gene_ids")</code> attribute is set.
species_from	Character vector of the same length as <code>input_dfs</code> ; species label for each dataset (currently supports "human" and "mouse").
ref	Character; reference species for the common gene space (default "human").

Details

Align multiple MCMC outputs to a common set of homologous genes

Value

A list of the same length as `input_dfs`, each element row-filtered to the intersection of homologous reference-space gene IDs and reordered consistently.

Examples

```
## Not run:
aligned <- match_homologs(list(human_res, mouse_res),
                           species_from = c("human", "mouse"))

## End(Not run)
```

merge_mcmc	<i>Merge MCMC output lists by orthologs</i>
------------	---

Description

Intersects gene sets across a list of MCMC outputs from different species or conditions. When `ortholog.db` is supplied and multiple species are present, the ortholog database maps each species' genes to a common reference gene space (defined by the reference dataset). When datasets share a single species (or no ortholog database is provided), the function simply takes the intersection of gene names. If `uniqG = TRUE`, duplicate ortholog matches are resolved by keeping the entry with the highest mean absolute posterior signal.

Index

`bfdr_from_posterior`, [10](#)

`CB_getAllEst`, [6](#)
`CB_init_single`, [1](#)
`CB_MCMC_single_rj_slice`, [3](#)
`CBt_init_single`, [2](#)
`CBt_sim_data`, [7](#)
`circular_HDI`, [8](#)
`circular_median`, [9](#)

`detect_rhy`, [11](#)

`match_homologs`, [21](#)
`match_symbols`, [9](#)
`merge_mcmc`, [21](#)
`multi_conservation`, [19](#), [20](#)
`multi_conservation_pathway`, [17](#)
`multi_conservation_pathway_bootstrap`,
[19](#)

`pathSelect`, [15](#)
`phase_infer`, [14](#)
`plot_pathway_integrated`, [16](#)

`summarize_bay`, [12](#)

`transition_classify`, [12](#)
`transition_classify_marginal`, [13](#)